

Teaching Monocular Depth the Metre with the Phone’s Own LiDAR: Fine-Tuned vs. Pretrained Depth Anything V2 for Room Reconstruction, Measured Against a Laser Scanner

Author Name
Affiliation
email@example.com

Abstract—Devices without LiDAR can reconstruct a room from monocular depth, but an off-the-shelf “metric” model such as Depth Anything V2 Metric-Indoor, trained on synthetic images, misplaces a real room by 54.3 cm inside a reconstruction pipeline. We ask whether *transfer learning with depth from the phone’s own LiDAR*—no laser scanner involved—really makes this model better than its pretrained self *when measured against a laser scanner*, the Faro ground truth that ARKitScenes ships. We fine-tune only the DPT head (frozen encoder) on 24 ARKitScenes rooms with labels from the iPad’s LiDAR (R3) and compare it with the pretrained checkpoint through one depth $\rightarrow$ TSDF $\rightarrow$ mesh pipeline in which the depth source is the single variable switched from a config file, on rooms never used for training or checkpoint selection: six rooms with a Faro reference mesh (3D and 2D) and 20 further rooms from the Validation fold (per-frame 2D against Faro depth, 2,090 frames). R3 lowers Chamfer from 54.3 cm to 15.7 cm ($3.5\times$, better in every room), raises $F@5$ cm from 0.03 to 0.24 and lowers per-frame AbsRel from 0.382 to 0.134; on the 20 unseen rooms AbsRel falls from 0.410 to 0.130 (better in 19/20)—all without calibration or any sensor at run time. A laser teacher (14.6 cm) is not measurably better than the LiDAR teacher, and Depth Pro given the camera’s true focal length, without fine-tuning, reaches 155.4 cm. The same pipeline gives the context: it loses 2.1 cm on its own, iPad LiDAR 3.0 cm, a monocular model with the correct scale per frame 5.3 cm, and a per-frame fit on ~ 200 sparse metric points 6.0 cm. Per-frame analysis shows R3’s remaining error is a scale that swings with the content of each view, which fine-tuning de-biases but does not remove. Supervision that used to need a laser scanner can thus be collected with a consumer depth sensor and used to train a model that needs no sensor at run time (measured here on the same iPad camera); the resulting rooms (~ 16 cm) are a large step toward, but not yet at, the accuracy a rough overview needs, let alone furniture placement. Code, every experiment config, every run’s metrics, checkpoints and the room split are released.

Index Terms—monocular depth estimation, transfer learning, metric depth, LiDAR supervision, TSDF fusion, 3D room reconstruction, ARKitScenes

I. INTRODUCTION AND BUSINESS MOTIVATION

A. Problem

Scanning a room into a 3D model is already a consumer feature on iPhone Pro and iPad Pro through Apple RoomPlan and apps such as Polycam, all of which rely on the *LiDAR*

sensor for directly metric depth. Devices with LiDAR are, however, a minority of the market: non-Pro iPhones and almost every Android phone have none. The alternative for these devices is *monocular depth estimation*—networks that predict depth from a single RGB image—which advanced rapidly between 2020 and 2024 (MiDaS, Depth Anything) to the point of producing visually convincing results on single images.

The question a product team must answer before investing is: **if LiDAR is replaced by a monocular model while the rest of the system stays the same, by how many centimetres is the reconstructed room wrong, and which use cases does that still serve** (floor-area estimation, furniture placement, renovation measurement)? Monocular-depth research reports per-image metrics (AbsRel, δ_1) on benchmarks, which do not translate into “how crooked is the wall.” End-to-end reconstruction systems (Atlas, NeuralRecon, SimpleRecon) design their whole pipeline around their own model, so the *depth source* cannot be compared in isolation.

Some recent models claim to output *metres* directly, without calibration (DA-v2 Metric-Indoor, ZoeDepth, Depth Pro)—if that held, the question above would reduce to choosing a model. These models, however, are trained on data that are not a phone’s camera (DA-v2 Metric-Indoor on rendered Hypersim images), and inside a real room-reconstruction pipeline the room comes out 54.3 cm wrong (§V-C). The direct remedy is *transfer learning*: keep training the model on depth from the target domain. The question is where those labels come from—the \$20k laser scanner used to build research datasets, or the *LiDAR already inside Pro phones*, which an app maker can collect at scale. If the latter works, the minority of devices with LiDAR can be the teacher of the majority without it.

B. Research question

Main question: is DA-v2 Metric-Indoor fine-tuned with depth from the iPad’s LiDAR (no laser scanner at training time) *better than the same model pretrained*, when measured against the Faro laser scanner—the raw ground truth ARKitScenes ships—both as a room mesh (centimetres on the wall)

and as per-frame depth, on rooms never used for training?
Supporting questions: (a) how much better is a laser teacher than a phone-LiDAR teacher; (b) where does the fine-tuned model stand relative to using a sensor at run time (LiDAR, sparse points per frame); (c) where does the remaining error come from?

C. Approach

This is a *system-development* study. We build a classical room-reconstruction pipeline (per-frame depth $\rightarrow$ scale alignment $\rightarrow$ TSDF voxel fusion $\rightarrow$ mesh) in which **the depth source is the single swappable variable**, so “pretrained” and “fine-tuned” are two rows that differ only in a checkpoint name, and every other depth source can be priced through the same pipeline as context:

Fine-tuning uses ARKitScenes rooms disjoint from the test rooms at the `visit_id` level and trains only the DPT neck and metric head. Evaluation uses Faro twice: a reference mesh fused from Faro depth on six test rooms (3D), and Faro depth per frame directly on those rooms and on 20 more from the Validation fold (2D). Isolating *ScaleAligner* as a separate stage further lets us separate “wrong shape” from “wrong scale”: `oracle_frame` (fit per frame against GT) is a model’s ceiling, `per_scene` is a one-off calibration, and `sparse_points` (fit per frame on a few hundred metric points) is what a system with a VIO tracker can do.

D. Main results

- **The LiDAR-teacher fine-tune (R3) beats the pretrained model in every test room:** Chamfer 54.3 cm $\rightarrow$ 15.7 cm (3.5 $\times$, 6/6 rooms), F@5 cm 0.03 $\rightarrow$ 0.24, per-frame AbsRel against Faro 0.382 $\rightarrow$ 0.134—with no calibration and no sensor at run time (§V-A).
- **The same holds on 20 rooms that took part in nothing:** AbsRel 0.410 $\rightarrow$ 0.130, better in 19/20 rooms (exact Wilcoxon $p = 3.8 \times 10^{-6}$) (§V-A).
- **A laser teacher is not measurably better than a phone-LiDAR teacher:** 14.6 cm (Faro) / 16.6 cm (LiDAR, 10 rooms) / 15.7 cm (LiDAR, 24 rooms), within the across-room std (§V-B).
- An intrinsics-conditioned metric model (Depth Pro), given the real focal length and no fine-tuning, reaches only 155.4 cm—knowing the camera does not replace in-domain labels (§V-B).
- **R3 does not reach a sensor:** the pipeline itself loses 2.1 ± 0.5 cm, iPad LiDAR 3.0 ± 0.6 cm, monocular depth with the correct scale on every frame 5.3 ± 1.3 cm, and a per-frame fit on ~ 200 sparse points (VIO proxy) 6.0 cm (§V-C).
- **The remaining error is scale that swings per view, not shape:** one calibration per room gives 22.9 ± 2.1 cm uniformly; the model’s scale varies several-fold within one scan with the content of the frame, not with time (§VI). Fine-tuning removes the domain bias; the per-frame factor still has to come from metric data at run time.

- A 13 $\times$ smaller model (DA-v2 Small) costs 1.0 cm for a 6 $\times$ speed-up, and frame count governs coverage, not accuracy (≥ 0.8 fps is needed).

E. Contributions

- 1) **A measured answer to “does fine-tuning on a phone’s own LiDAR really help a metric monocular model?”**, against a laser scanner on held-out rooms in 3D (6 rooms) and 2D (6 + 20 rooms), with the checkpoints, the room split and the room-selection scripts released (§IV-E, §V-A).
- 2) A teacher ablation (Faro vs. LiDAR, 10 vs. 24 rooms) and an intrinsics-conditioned baseline under one training recipe (§V-B).
- 3) An open-source pipeline in which *DepthSource* and *ScaleAligner* are interfaces switched from config without touching the orchestrator, so every table is “one loop, one variable” and a newly trained checkpoint becomes a new row immediately (§III).
- 4) “Centimetres on the wall” for Faro / LiDAR / monocular $\times$ scale strategy through one pipeline, and an analysis showing that monocular error comes from per-view scale, not temporal drift—which explains both why fine-tuning helps and why it is not enough (§V-C, §VI).
- 5) Per-scan cost (time, frames, model size, LiDAR confidence weighting) for business decisions (§VII).

a) What this paper does not claim: It does not claim to beat Atlas / NeuralRecon / SimpleRecon—they are not compared, and this pipeline is not designed to compete. It does not claim that fine-tuning makes LiDAR unnecessary: R3 (15.7 cm) remains far from LiDAR (3.0 cm) and from having metric points at run time (6.0 cm). It does not claim that a LiDAR teacher equals or beats a laser teacher—R1 and R2 differ in both label and number of training views (§IV-E) and by less than the across-room std. Only the head of one model is fine-tuned (frozen encoder, ARKitScenes, iPad camera); every other model is used pretrained, and other phone cameras are not measured.

II. RELATED WORK

A. Monocular depth estimation

Monocular depth models fall into two families according to what their output promises.

a) Relative / affine-invariant: MiDaS [1] introduced training across many datasets with a loss that ignores scale and shift, giving strong cross-domain generalisation at the price of an output that is *disparity up to an affine transform*: two constants (s, t) per image must be known to obtain metres. Depth Anything [2] and Depth Anything V2 [3] extend the idea with large unlabelled corpora and synthetic teachers, giving much sharper edges and detail while remaining affine-invariant in disparity. We use DA-v2 (Large / Small) and MiDaS small as representatives of this family and **treat (s, t) as an experimental variable** (the *ScaleAligner*) rather than hiding it in the model.

Depth source	Stands for
DA-v2 Metric-Indoor pretrained / fine-tuned with a LiDAR teacher (R3)	main question: a device with no sensor and no calibration
Faro laser (<code>highres_depth</code>)	upper bound of the pipeline: its own loss with perfect depth
ARKit LiDAR (<code>lowres_depth</code> , 256×192)	what iPhone Pro does today (and R3’s teacher)
Monocular (Depth Anything V2, MiDaS) $\times$ scale strategy	ceiling and cost of supplying scale at run time instead of training

b) Metric: ZoeDepth [4] and the metric variants of DA-v2 (fine-tuned on Hypersim / Virtual KITTI) aim to output metres directly. Metric3D [5] identifies the camera focal length as the main ambiguity and resolves it by normalising images to a canonical camera. These works report per-image AbsRel/δ_1 on NYUv2/KITTI; our results (§VI-C) show that on real room video the per-frame scale of a metric model still swings from -60% to $+80\%$ within one scan—invisible to per-image metrics, immediately visible once frames are fused into one volume.

c) Conditioning on the camera, or on a teacher: Two families answer the scale ambiguity differently. One conditions the network on camera intrinsics—Metric3D [5], UniDepth [6], and Depth Pro [7], which estimates the focal length itself—so a metre prediction is tied to a known field of view. The other adapts a pretrained model to the target domain with depth labels, which is what the metric DA-v2 variants do on Hypersim. We take the second route but change where the labels come from: instead of a laser scanner or a synthetic renderer, the teacher is the depth sensor already shipping in consumer phones. This puts a price on the teacher—the question a product team actually faces—rather than on the architecture, and it is measured against a laser scanner both in mesh centimetres on held-out rooms and in per-frame AbsRel on 26 rooms, not on a single-image benchmark (§V-A). The two families are complementary: an intrinsics-conditioned model could be fine-tuned with the same LiDAR labels. We measure one of them (Depth Pro, handed the camera’s true focal length) as the no-fine-tuning baseline row of Exp. 6: on these captures it does *not* deliver usable metric depth (§V-B), which is why fine-tuning is the axis we study.

d) Mobile LiDAR depth: ARKit provides 256×192 depth fusing LiDAR with imagery; ARKitScenes [8] is the first dataset to release both this depth and Faro laser-scanner depth for the same rooms, making a direct “mobile LiDAR vs. laser” comparison possible. We use both as two rows of the same table.

B. Multi-view 3D reconstruction from video

a) Classical depth fusion: Volumetric TSDF fusion [9] and KinectFusion [10] combine many depth frames through a truncated signed distance per voxel and extract the surface with marching cubes. The property we need is that **fusion neither knows nor cares where depth comes from**—as long as it is metric and posed—so it is a fair “fixed part” for every source. We use the Open3D implementation [11].

b) End-to-end learned reconstruction: Atlas [12] regresses a TSDF directly from multi-image features, Neural-

Recon [13] does so incrementally in real time, and SimpleRecon [14] shows that good multi-view depth plus classical fusion is competitive. These are strong ScanNet baselines, but all learn depth *from multiple images*; they do not answer “what room do you get from a per-image monocular model plus the device’s VIO pose,” which is the situation of an app on a device without LiDAR. We therefore **do not compare against this group** and make no claim of superiority.

c) Scale in monocular SLAM / reconstruction: Using monocular depth inside SLAM requires per-frame scale recovery, e.g. from sparse tracked points (CNN-SLAM [15] and successors), or joint optimisation of scale/shift with pose (MonoSDF [16] uses monocular priors in a neural surface). Our `sparse_points` aligner follows this line; we evaluate it next to `oracle_frame` and `per_scene` so that the “price of not knowing scale” is measured at three levels of available metric information.

C. Room-scanning products

Apple RoomPlan [17] produces a floor plan and furniture boxes from LiDAR on iPhone/iPad Pro; Polycam and similar apps produce meshes or photogrammetry. All require LiDAR for room mode or fall back to slow offline photogrammetry with many photos. No public report states how many centimetres these products deviate from a laser scanner; this paper provides that number at least for our own pipeline.

D. Position of this work

III. SYSTEM DESIGN

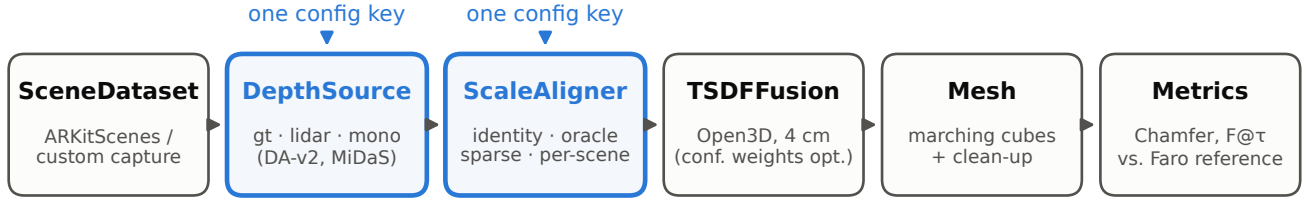
A. Design principle: one pipeline, one variable

The research question (“change only the depth source—how much changes?”) imposes three architectural requirements:

- 1) **Every depth source must pass through the same code after the point where it is produced**—no special branch for mono or LiDAR in fusion or evaluation.
- 2) **Whatever can be swapped is swapped from config**, not by editing code: one result table = one YAML file.
- 3) **The paper’s code is the product’s code**: the same system must accept ARCore depth or a smaller model on the day an MVP ships, without a rewrite.

Requirement 1 leads to the `DepthSource` interface; requirement 2 to a `string`→`class` registry and `base-plus-override` configs; requirement 3 to a single monolithic package with no service, database or experiment tracker.

	Trains	Compares depth sources	Reference	Metrics
Monocular depth papers	✓	× (one model)	per-image depth	AbsRel, δ
Atlas / NeuralRecon / SimpleRecon	✓	× (multi-view only)	ScanNet mesh	Chamfer, F-score
This work	head only (LiDAR teacher)	✓ laser / LiDAR / mono × aligner × teacher	Faro laser (ARKitScenes)	Chamfer, F@ τ , per-frame AbsRel/ δ , time



frames: RGB + pose (VIO) + optional sensor depth / confidence / sparse points

Fig. 1. Pipeline components. The two highlighted boxes are the only things that change between rows of a result table; everything to their right is fixed (Open3D ScalableTSDFVolume, marching cubes, cluster filtering). Frame stride (Exp. 3) is a dataset parameter; voxel size (Exp. 2) is a fusion parameter.

Point	Type	Owner
Frame{rgb, pose_c2w, gt_depth?, extra}	from SceneDataset	dataset mapping
pred: (H,W) float32, relative <i>or</i> metric	DepthSource.get_depth(frame)	source declares is_metric
depth_m: (H,W) float32 metres, 0 = invalid	ScaleAligner.align(pred, frame)	invariant: metric after the aligner

B. Overview

Fig. 1 shows the data flow. Per frame there are three typed hand-over points:

Invariants enforced in code, not just documented: (i) values are metres after the aligner; (ii) pose_c2w is camera-to-world and is inverted in exactly one place (TSDFFusion.integrate); (iii) intrinsics are bound to a resolution—resizing requires `Intrinsics.scaled()`; (iv) `is_metric=True` $\Rightarrow$ `aligner=identity` and `is_metric=False` $\Rightarrow$ `aligner!=identity`—the pipeline fails at construction if the pair is wrong.

C. DepthSource: depth as a strategy

```

class DepthSource(ABC):
    name: str
    is_metric: bool
    def get_depth(self, frame) -> np.ndarray
        # (H,W) float32; 0/NaN = invalid

```

It takes the whole `Frame` rather than an RGB image because each source needs different inputs: `gt` reads `frame.gt_depth`, `lidar` reads `frame.extra["lidar_depth"]` (plus confidence), `mono` reads `frame.rgb`. One signature covers all cases without edits when `arcore` is added. `MonocularDepth` does three things that belong neither to the model nor to the aligner: it rotates the image upright according to `sky_direction`, converts disparity to $1/x$ so that “larger = farther,” and masks invalid values. Models (`DepthModel: DA-v2, MiDaS`) form a second registry under `mono`, so Exp. 4 (model size) is just `depth.model=...`

D. ScaleAligner: scale as a separate stage

A relative model gives depth up to an affine transform. How (s, t) is obtained is a **system decision** that should be measured separately from the model, so it is its own stage: The gap `oracle_frame` $\rightarrow$ `per_scene` is “the price of not knowing scale”—an experimental result, not noise; `sparse_points` measures how much of that price a phone’s tracker can pay back.

a) *Fitting space*: Affine-invariant models are affine in *disparity*, not depth; fitting $gt \approx s \cdot pred + t$ in depth space is the wrong model. We therefore fit $1/gt \approx s \cdot (1/pred) + t$ (least squares with three rounds of trimming the worst 20% of residuals) and invert back. On scene 42444474 this change alone moved `mono+oracle` from 15.2cm to 5.4cm—an example of why the aligner must be testable in isolation.

b) *Sparse points*: ARKitScenes ships no ARKit feature points, so `sparse_points` is evaluated against a proxy: $N=200$ pixels sampled per frame (deterministically) from the LiDAR frame at confidence 2, which is what a tracker hands the pipeline—a few hundred metric depths per frame, nothing dense. The fit is per frame in inverse depth with two rounds of 10% trimming; frames with fewer than 20 usable points reuse the previous frame’s (s, t) , as a deployed system would when tracking momentarily loses features. The proxy is optimistic in one respect (LiDAR points are cleaner than triangulated VIO features) and pessimistic in another (VIO points cluster on texture, ours are uniform); we treat the row as an *upper bound of real VIO*.

Aligner	Fits (s, t)	Meaning
identity	—	GT / LiDAR / metric models
oracle_frame	per frame, against GT depth	model ceiling (not deployable)
per_scene	once, on N frames spread over the scan, then frozen	one-off calibration
sparse_points	per frame, against ~ 200 sparse metric points	the MVP path (VIO feature points)

E. Fixed parts: fusion, post-processing, evaluation

- **Fusion:** Open3D ScalableTSDFVolume, 4 cm voxels, `sdf_trunc = 3 × voxel`, depth 0.1–5 m; every source is resampled to 256×192 (the LiDAR resolution) before integration for fairness, and only pixels where the reference has depth are integrated (`mask_to_gt`)—no source is penalised for seeing beyond the Faro coverage. An optional per-pixel integer weight from the sensor’s confidence map (`fusion.confidence_weights`) is off for every main result and used only in Exp. 5.
- **Post-processing:** small clusters removed; no decimation.
- **Reference:** ARKitScenes provides no laser mesh; we fuse Faro `highres_depth` from every frame at 1 cm voxels once per scene into `reference_mesh.ply`. The gt row at 4 cm therefore measures “pipeline loss,” not zero.
- **3D metrics** (fixed protocol in `metrics_3d.py`): 200k points sampled on both meshes (fixed seed); accuracy = mean `pred → ref` distance, completeness = `ref → pred`, Chamfer = their mean; precision/recall/F-score at 2/5/10 cm (**F@5 cm is the headline**); normal consistency = $\text{mean } |n_{\text{pred}} \cdot n_{\text{ref}}|$.
- **2D metrics** per frame (RMSE, AbsRel, δ_{1-3}) compare depth *after* alignment with the dataset’s Faro depth, to tell whether error arises before or after fusion, and are Exp. 7’s measure; pixels are chosen by the ground truth and predictions clipped (protocol 2, §IV-F). `roomscan eval2d` runs this part alone (no fusion) for rooms that have no reference mesh.

F. Config, registry and experiments

`configs/base.yaml` defines every value; `configs/experiments/expN.yaml` = base + a list of runs, each a handful of dotted-key overrides. `roomscan sweep` (or `eval2d`, 2D only) runs every run $\times$ every scene and writes a resolved `config.yaml` plus `metrics.json` per run; `roomscan report` aggregates them into `summary.csv` and one `table.md` per experiment. The tables in §V are copied from those files.

Every kind of extension is one new file plus one registry line, without touching `pipeline.py`: a depth source (AR-Core), a model (a compressed variant), an aligner, a dataset (a real room captured by the app; `dataio/custom.py` reads the app’s folder format), or an experiment. The rule that guards the design: if adding something requires editing `pipeline.py`, stop and write an architecture decision record. Over the project this happened exactly once (per-pixel confidence weights, two lines), and it is recorded as such.

G. Deliberately not done

GPU TSDF, an experiment tracker, and pose estimation (the dataset’s ARKit VIO poses are used) are out of scope; a minimal web service and viewer exist as a separate package importing the pipeline, and none of these affect the numbers.

IV. EXPERIMENTAL SETUP

Main question. Is DA-v2 Metric-Indoor fine-tuned with the iPad’s own LiDAR (R3) better than the same model pretrained, when measured against the *Faro laser scanner, the raw ground truth ARKitScenes ships*? Exp. 6 answers it on six test rooms in 3D and 2D, Exp. 7 on 20 further rooms in 2D. Exp. 1–5 price the other depth sources through the same pipeline as context for where monocular error comes from and where R3 stands relative to a sensor.

A. Dataset: ARKitScenes

We use ARKitScenes [8] because it is the only dataset that provides **all three** things the question needs for the same room: RGB video from a real device (iPad Pro), depth from that device’s LiDAR (`lowres_depth`, 256×192), and depth from a Faro laser scanner registered to the camera poses (`highres_depth`, 1920×1440). The “iPhone Pro today” row and the reference therefore come from one scan, with no cross-sensor alignment of our own.

Ground truth in this paper is the dataset’s Faro data as distributed, not corrected or interpolated: in 2D we compare per-frame depth with `highres_depth` directly (nearest-neighbour down-sampling to the 256×192 grid, so every GT value used is a value Faro measured, never an average); in 3D we compare with a reference mesh fused from that same `highres_depth` (§IV-B), because the dataset ships no laser mesh. The iPad LiDAR (`lowres_depth`) is *never* used as ground truth—only as the teacher of R2/R3 and as the `lidar` row.

Two dataset constraints shape the experiments: (1) Faro depth exists only for the frames Apple kept (~ 4 fps), so every source is evaluated **on this same frame set** and Exp. 3 (frame stride) measures coverage more than compute (§V-E); (2) no laser mesh is provided, so we build one (§IV-B).

Scenes (Table I) were shortlisted from metadata without downloading: `highres_depth` must exist, the video must be in the 3DOD split, and the set must span room types (standard / large / glossy-mirror) that are the expected failure cases of monocular depth. Scene 42444474 was used during development, so its numbers are not held-out; we report per-scene tables in §V to show that the conclusions hold on the five scenes the pipeline never saw.

Asset	Resolution / rate	Role
vga_wide RGB	640 × 480, 30 fps	input to monocular models (rotated upright)
lowres_depth + confidence	256 × 192, 60 fps	depth source lidar; sparse-point proxy; Exp. 5 weights
highres_depth (Faro)	1920 × 1440, ~4 fps (2.7–5.1 per scan)	depth source gt and reference mesh
lowres_wide.traj	~10 Hz	pose (ARKit VIO), slerp/lerp-interpolated to each frame
lowres_wide_intrinsics	per frame	scan median, scaled to resolution

TABLE I

SCENES. EXTENT = AXIS-ALIGNED EXTENT OF THE REFERENCE MESH (WHAT FARO SAW); FARO FRAMES = NUMBER OF FRAMES EVERY ROW IS EVALUATED ON. TOTAL: 2,352 FRAMES / 577 S OF VIDEO.

video_id	Room (3DOD labels)	Extent (m)	Scan (s)	Faro frames	Role
42444474	open-plan kitchen/dining under sloped roof, glass table/rail, brick wall	9.9 × 5.1 × 4.2	88	236	large + glass (dev. scene)
47333774	standard bedroom (bed, shelf, cabinet, table)	5.1 × 4.9 × 3.3	90	375	standard #1
47115299	large living room (4 tables, 2 sofas, TV)	8.1 × 6.1 × 2.5	91	324	large
42897743	bathroom (bathtub, sink, toilet)	8.0 × 3.4 × 3.6	114	448	mirror / tiles
47429736	bedroom + closets (bed, 5 cabinets, 2 tables)	6.6 × 6.4 × 2.7	98	504	standard #2
45261556	kitchen (10 cabinets, stove, sink)	7.1 × 6.1 × 2.4	98	465	glossy #2

B. Reference mesh

We fuse `highres_depth` from **every** frame with TSDF at 1cm voxels (`sdf_trunc 3cm`) once per scene into `reference_mesh.ply`, and never touch it again. Consequences for reading the tables: the `gt` row runs at the experiment’s voxel size (4cm) and therefore has non-zero Chamfer—the **discretisation loss of the pipeline itself**, a lower bound for every row; the reference covers only what Faro saw, so every source integrates only pixels where Faro has a value (`eval.mask_to_gt`); and the reference uses the same poses as every row, so VIO pose drift is **removed from the comparison by design**—all numbers are the effect of the depth source alone (limitation: §VIII).

C. Pipeline configuration (fixed for every row)

Monocular models (pretrained, except the fine-tuned rows of Exp. 6–7, §IV-E): Depth Anything V2 Large (335M parameters) / Small (25M) and DA-v2 Metric-Indoor (Large, fine-tuned on Hypersim); MiDaS v2.1 small (21M) for Exp. 4. Every model receives the upright image; its output is converted to depth-like (1/disparity) before the aligner.

D. Scale-alignment protocol

Every non-identity aligner fits an affine map **in inverse-depth space**: (s, t) such that $1/gt \approx s \cdot (1/pred) + t$, by trimmed least squares—three rounds, dropping the worst 20% of residuals each round—on pixels where both GT and prediction are valid within 0.1–5m, then inverted back to metres. `per_scene` also uses GT to fit, so it is the **upper bound of one-off calibration**: a real system that calibrates from a ruler or a few VIO points will do no better.

E. Fine-tuning protocol (Exp. 6)

a) Scenes: Disjoint at the `visit_id` level from the six test scenes, drawn with seed 0 from the ARKitScenes screening list: 10 *training* scenes that have both a Faro scan and LiDAR (so R1 and R2 train on the same *rooms*), 14

further training scenes with LiDAR only (no laser scan exists for them), and 3 scenes from the official Validation fold, with Faro, used for checkpoint selection. Six test scenes, 3 validation, 24 training; no room, and no other capture of the same room, appears in two of those sets. The split file (`configs/training/splits.yaml`) and the per-scene frame counts are released with the code.

b) Labels: R1 uses Faro `highres_depth` (3594 frames over 10 scenes); R2 uses the iPad’s `lowres_depth` with ARKit confidence ≥ 1 on the same 10 scenes (8656 frames); **R3, the paper’s main model**, uses the same LiDAR labels over all 24 scenes (19 896 frames). R1’s and R2’s frames are *not* the same set: R1 iterates the Faro timestamps (~4 fps) while R2/R3 iterate every third `vga_wide` frame (~10 fps), because LiDAR accompanies every colour frame; at equal steps R2 therefore sees ~2.4× more distinct views than R1 (read §V-B with this in mind). Depth is resampled to the RGB grid with nearest-neighbour only; pixels beyond 5 m or without a label are masked out.

c) Optimisation: The DINOv2 encoder is frozen (300M parameters in `eval` mode); only the DPT neck and the metric head train, with SiLog loss ($\beta = 0.15$), AdamW (lr 5×10^{-5} , weight decay 0.01, 200 warm-up steps then cosine), bf16 autocast, batch 2 with gradient accumulation 4, on 518×518 crops of images resized so the short side is 518. R1 and R2 run 6000 steps, R3 12 000; validation every 500 steps keeps the checkpoint with the lowest AbsRel on the 3 validation scenes. A 6000-step run takes about an hour on an RTX 3070 (3.2 GB of VRAM), R3 about two.

d) At test time: the fine-tuned models are plain DepthSource rows like any other: RGB in, metres out, aligner: `identity`, no access to depth, poses or intrinsics. Input preparation is identical for the pretrained and fine-tuned rows (upright rotation, short side 518 and long side rounded to a multiple of 14, i.e. 518×686 , ImageNet normalisation) and matches the training resolution, so the

Part	Value	Reason
fusion resolution	256×192	LiDAR resolution; every source resampled to it
voxel / sdf_trunc	4 cm / 12 cm	Exp. 2 shows 4 cm is the sweet spot
depth range	0.1–5.0 m	cuts near sensor noise and far Faro noise
post-processing	drop clusters < 1000 triangles	no decimation
frame stride	1 (every Faro frame)	except Exp. 3
confidence weights	off	except Exp. 5
device	Apple M-series (MPS)	reported times are on this machine

Aligner	Fitted on	Interpretation
oracle_frame	that frame’s GT depth, every frame	model ceiling (upper bound)
per_scene	GT depth of 10 frames spread over the scan , stacked, fitted once; one (S, T) for all frames	upper bound of a one-off calibration
sparse_points	200 LiDAR pixels/frame at confidence 2 (VIO proxy), per frame; 2 rounds of 10% trimming; < 20 points $\Rightarrow$ reuse previous (s, t)	upper bound of a VIO-based system

difference between the two rows is the head’s weights alone.

F. Metrics

a) *3D (headline)*: Following `metrics_3d.py` exactly for every row: 200,000 points sampled uniformly on the produced mesh and on the reference (seed 0); accuracy = mean pred $\rightarrow$ ref distance (did we build something that is not there?), completeness = ref $\rightarrow$ pred (did we miss something?), **Chamfer = their mean**; precision/recall/F-score at $\tau = 2/5/10$ cm—**F@5 cm is the headline** (5 cm $\approx$ the tolerance for furniture placement); normal consistency = mean $|n_{pred} \cdot n_{ref}|$ over nearest pairs.

b) *2D*: Per frame, depth *after* alignment vs. Faro `highres_depth` on the 256×192 grid: RMSE, AbsRel, δ_1 (fraction with $\max(p/g, g/p) < 1.25$). *Protocol 2* (the standard NYU/KITTI definition): pixels are selected by the ground truth alone ($0.1 \text{ m} \leq \text{GT} \leq 5 \text{ m}$) and by whether the source produced a value (a LiDAR pixel masked by confidence is not a prediction); the prediction is then *clipped* to 0.1–5 m rather than dropped. An earlier draft also dropped pixels whose *prediction* left the range, which hides the largest errors of a model that over-estimates distance; every 2D number here was recomputed with protocol 2. Every Exp. 1–6 row moved by less than 0.001 except Depth Pro, whose AbsRel rose from 1.04 to 2.007 because it often predicts beyond 5 m—exactly the case the old definition hid. Pixel-weighted within a room, then averaged over rooms. In Exp. 1–5 the 2D metrics separate depth error from fusion error; in Exp. 6–7 they are the most direct measure of how right a model’s distances are against the laser.

c) *Time*: `time_total_s` = data loading + inference + alignment + fusion + extraction on one machine; inference is reported separately in §VII.

G. Experiments

a) *Exp. 7*: The six test rooms are the only ones with a reference mesh, but 2D evaluation needs only the dataset’s Faro depth. We therefore add 20 rooms from the ARKitScenes Validation fold (`configs/eval/valfold_`

`2d.yaml`, `scripts/select_valfold_2d.py`, seed 0): scans of 30–150 s with ≥ 40 MB of `highres_depth`, one video per `visit_id`, excluding every visit that belongs to the fine-tuning train, validation or test sets or that also appears in the Training fold. These rooms decide nothing (checkpoints were already chosen on the three validation rooms). They are evaluated with `roomscan eval2d`, which uses the same frames, grid, mask and `depth_metrics` call as the pipeline without fusing (tested to reproduce the full pipeline’s numbers), on every second Faro frame (~ 2 fps).

Every run is one resolved `config.yaml` plus `metrics.json` under `experiments/results/<exp>/<scene>_<run>/` (all committed). Tables in §V are generated by `roomscan report` without manual editing; numbers are mean $\pm$ std over the six scenes, with per-scene tables in `per_scene.md`. Everything reproduces with `make reference`, `make sweep-exp1 ... sweep-exp6`, `make eval-exp7`, `make report` after downloading the data.

V. RESULTS

The main result is §V-A: DA-v2 Metric-Indoor fine-tuned with the iPad’s own LiDAR as the teacher (R3) against the same checkpoint pretrained, both measured against the Faro laser scanner—the raw ground truth that ARKitScenes ships. §V-B ablates the teacher; §V-C–V-G (Exp. 1–5) price every other depth source through the same pipeline and serve as context for where the remaining error comes from. 3D numbers are copied from `experiments/results/<exp>/table.md` (mean $\pm$ std over scenes) and `per_scene.md`, generated by `roomscan report`; macros are generated by `scripts/paper_numbers.py`. Every 2D number uses protocol 2 (pixels selected by the ground truth, prediction clipped; §IV-F). Units are cm; F is the F-score at $\tau = 5$ cm.

A. Main result: LiDAR-teacher fine-tune (R3) vs. the pre-trained model

Question. If a metric monocular model is taught with depth from the iPad’s own LiDAR—no laser scanner

Exp.	Variable	Values	Fixed
1 depth source	depth.source × depth.aligner	gt / lidar / mono+oracle / mono+sparse / mono+per_scene / metric+identity	DA-v2 L, 4 cm, stride 1
2 voxel size	fusion.voxel_size	2 / 4 / 8 cm (sdf_trunc = 3×)	gt
3 frame stride	dataset.frame_stride	1 / 5 / 10 / 20 × {gt, mono+oracle, mono+per_scene}	DA-v2 L, 4 cm
4 model size	depth.model	DA-v2 L / DA-v2 S / MiDaS small	oracle_frame, 4 cm
5 LiDAR confidence	lidar_min_confidence, confidence_weights	all / mask _{≥1} / mask=2 / weights [0,1,2] / [1,2,4]	lidar, 4 cm
6 fine-tune (main)	depth.model	pretrained / ft-LiDAR 24 rooms (R3); ablation: ft-Faro (R1) / ft-LiDAR 10 rooms (R2) / Depth Pro	identity, 4 cm
7 more rooms (2D)	rooms	pretrained / R3 / LiDAR on 20 Validation-fold rooms	identity, 2D only

in the training loop—is its depth really better than the same model pretrained, *when measured against the laser scanner*? The two rows differ only in the weights of the DPT neck and metric head: *pretrained* is Depth-Anything-V2-Metric-Indoor-Large (trained on synthetic Hypersim) and *R3* is that checkpoint fine-tuned on 24 ARKitScenes rooms with `lowres_depth` labels (§IV-E). At test time both see the same `vga_wide` frames, the same upright rotation and the same 518×686 input, with no alignment of any kind. No room in this subsection was used for training, checkpoint selection or any hyper-parameter (split by `visit_id`). We measure at two levels: *3D*, the mesh against the reference fused from Faro depth, on the six test rooms; and *2D*, per-frame depth against the dataset’s Faro `highres_depth` directly, on those six rooms *and on 20 further rooms from the ARKitScenes Validation fold* (Exp. 7, 2,090 Faro frames).

- 1) **The LiDAR-teacher fine-tune beats the pretrained model in every test room.** Chamfer 54.3 cm→15.7 cm (3.5×; 38.6 ± 8.6 cm better per room, 6/6 rooms, exact Wilcoxon $p = 0.031$, the smallest attainable at $n = 6$) and F@5 cm 0.03→0.24 (Tables II, III). The room where the pretrained model fails worst (kitchen 45261556) is still R3’s worst, but three times better.
- 2) **The 3D error falls because misplaced surfaces fall.** Accuracy (pred→ref) drops 70.3 cm→17.3 cm, more than completeness (38.3 cm→14.0 cm): the pretrained model builds the room but too far away (it over-estimates distance by 15–31 %, §VI-C); R3 brings the surfaces back near where they belong.
- 3) **The same holds in 2D on rooms that took part in nothing.** On the 20 Validation-fold rooms AbsRel drops 0.410→0.130 (3.2×; R3 better in 19/20 rooms, exact Wilcoxon $p = 3.8 \times 10^{-6}$) and δ_1 rises 0.333→0.827 (better in 19/20) (Table IV, Fig. 2). The one room that does not improve is 48018730 (AbsRel 0.275→0.281, δ_1

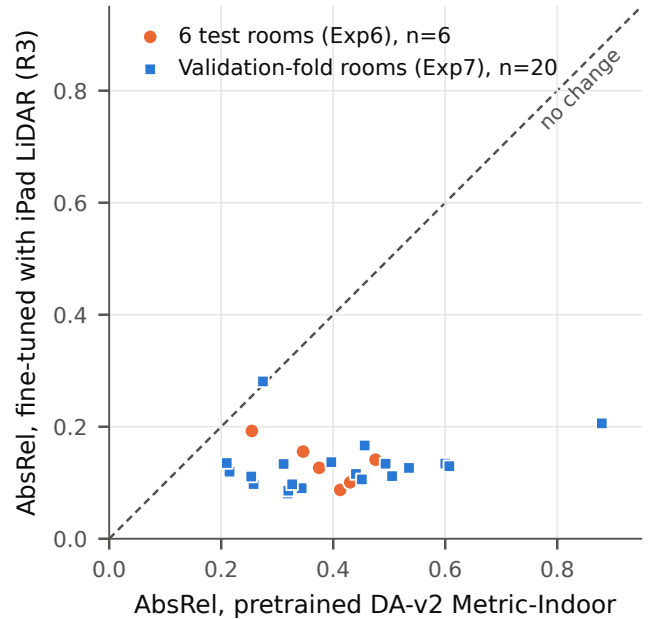


Fig. 2. Per-room AbsRel against Faro depth: pretrained (x) vs. the LiDAR-teacher fine-tune R3 (y), for the six test rooms (Exp. 6) and the 20 Validation-fold rooms (Exp. 7). Below the diagonal = the fine-tune is better.

0.53→0.35)—one of the pretrained model’s best rooms and R3’s worst; in the other 19, R3’s AbsRel is 21–64 % of the pretrained one (median ratio over all 20 rooms 0.27). The improvement is not a property of the six test rooms chosen at the start of the project.

- 4) **It does not reach a sensor.** R3 is still ~13 cm behind the iPad’s LiDAR in Chamfer and 9.7 cm behind a per-frame fit on 200 sparse points; in 2D on the Validation fold LiDAR has AbsRel 0.019 vs. R3’s 0.130. Fine-tuning fixes the scale of the *domain*; it does not fix the scale of the *frame being looked at* (§VI-A). For a device with

TABLE II

MAIN RESULT ON THE SIX TEST ROOMS: RGB ONLY, NO ALIGNMENT, MEASURED AGAINST FARO. THE TWO REFERENCE ROWS COME FROM EXP. 1 AND NEED A SENSOR AT RUN TIME.

Model	Teacher	Chamfer	Acc.	Compl.	F@5	R@10	AbsRel	δ_1	RMSE
DA-v2 Metric-Indoor, pretrained	— (Hypersim)	54.3 $\pm$ 10.6	70.3	38.3	0.03	0.10	0.382	0.277	49.3
+ fine-tune R3	iPad LiDAR, 24 rooms	15.7 $\pm$ 4.0	17.3	14.0	0.24	0.51	0.134	0.812	20.2
ref.: iPad LiDAR (sensor)	—	3.0 $\pm$ 0.6	2.2	3.9	0.93	0.97	0.018	0.997	3.9
ref.: DA-v2 L + 200 sparse pts/frame	—	6.0 $\pm$ 1.7	6.4	5.5	0.74	0.91	0.036	0.985	7.4

TABLE III

MAIN RESULT PER TEST ROOM (ORDER OF TABLE I): CHAMFER (CM) AND ABSREL.

	4244	4733	4711	4289	4742	4526	mean
Chamfer, pretrained	48.5	61.6	41.5	48.9	54.1	71.0	54.3
Chamfer, R3	12.8	16.7	17.3	12.2	12.4	22.5	15.7
AbsRel, pretrained	0.375	0.346	0.255	0.412	0.476	0.430	0.382
AbsRel, R3	0.127	0.156	0.193	0.087	0.141	0.101	0.134

TABLE IV

EXP. 7: 2D ONLY, 20 VALIDATION-FOLD ROOMS NEVER USED FOR ANYTHING ELSE (EVERY SECOND FARO FRAME, 2,090 FRAMES); MEAN $\pm$ STD OVER ROOMS. PER-ROOM VALUES:

EXPERIMENTS/RESULTS/EXP7_VALFOLD_2D/.

Row	AbsRel	δ_1	RMSE (cm)
DA-v2 Metric-Indoor, pretrained	0.410 $\pm$ 0.164	0.333 $\pm$ 0.216	50.9
+ fine-tune R3	0.130 $\pm$ 0.046	0.827 $\pm$ 0.135	18.4
ref.: iPad LiDAR (sensor)	0.019 $\pm$ 0.002	0.997 $\pm$ 0.002	3.8

neither LiDAR nor trustworthy VIO points, R3 is the only row in this paper that works without calibration at run time.

B. Teacher ablation and a no-fine-tuning baseline

R3 is one of three fine-tunes with the same recipe (frozen DINOv2 backbone, DPT neck + head, SiLog loss) that differ in label and number of rooms: Faro `highres_depth` on 10 rooms (R1, 3,594 frames), the iPad’s LiDAR on the same 10 rooms (R2, 8,656 frames), and LiDAR on 24 rooms (R3, the 10 + 14 rooms with no laser scan at all, 19,896 frames). R1 and R2 share rooms but *not frames*: Faro exists only on the frames Apple kept (~ 4 fps) whereas LiDAR accompanies every colour frame (every third taken, ~ 10 fps), so at the same 6,000 steps R2 sees $\sim 2.4\times$ more distinct views than R1. The R1–R2 comparison therefore changes both the label and view diversity. As a no-fine-tuning reference the table also carries Depth Pro [7], a metric model of the other family (§II): no fine-tuning, and *handed the camera’s true focal length* (533 px) rather than the one it predicts, which is the strongest form of that baseline.

- 1) **The laser teacher is not measurably better than the LiDAR teacher.** R1 14.6 cm, R3 15.7 cm and R2 16.6 cm all lie within the across-room std (4.0–4.7 cm), and no row wins every room (R1 beats R3 in three rooms, R3 wins two, one tie; Table VI). Given the frame difference above, the honest claim is that a phone’s depth

TABLE V

EXP. 6, ALL ROWS: SAME PIPELINE, SAME SIX SCENES, RGB-ONLY, NO ALIGNMENT. TEACHER = THE DEPTH USED AS THE TRAINING LABEL; NONE OF THESE ROWS USES A DEPTH SENSOR AT TEST TIME.

Model	Teacher	Chamfer	F@5	AbsRel	δ_1
Depth Pro (intrinsic given)	—	155.4 $\pm$ 39.0	0.00	2.007	0.170
Metric-Indoor (pretrained)	—	54.3 $\pm$ 10.6	0.03	0.382	0.277
+ fine-tune (R1)	Faro	14.6 $\pm$ 4.0	0.31	0.136	0.780
+ fine-tune (R2)	ARKit LiDAR	16.6 $\pm$ 4.7	0.23	0.136	0.848
+ fine-tune (R3)	LiDAR, 24 scenes	15.7 $\pm$ 4.0	0.24	0.134	0.812

TABLE VI

EXP. 6 CHAMFER (CM) PER SCENE; COLUMN ORDER AS IN TABLE VIII.

Teacher	4244	4733	4711	4289	4742	4526	mean
Depth Pro (no fine-tune)	131.1	165.2	172.9	196.4	177.9	88.9	155.4
none (pretrained)	48.5	61.6	41.5	48.9	54.1	71.0	54.3
Faro (R1)	10.8	13.8	17.6	21.1	12.4	11.9	14.6
LiDAR (R2)	13.9	19.0	12.8	12.5	16.6	24.6	16.6
LiDAR $\times 24$ (R3)	12.8	16.7	17.3	12.2	12.4	22.5	15.7

sensor can replace the laser scanner as the teacher at no measurable cost on these six rooms—not that the two are equal, or that either is better.

- 2) **Fourteen extra laser-free rooms do not help measurably** (R3 15.7 cm vs. R2 16.6 cm). R3 was the best run on the validation fold (AbsRel 0.133 vs. 0.142, measured during training under protocol 1) and is not the best here: three validation rooms are too few to rank checkpoints this close (§VIII). We take R3 as the main model for reasons that do not depend on the test numbers: it is the only recipe that scales without a laser scanner, and it was the best run on validation.
- 3) **Knowing the camera does not rescue metric depth.** Depth Pro, given the true intrinsics, is 155.4 cm with F@5 cm 0.00—worse than the pretrained DA-v2 Metric row it was meant to improve on, and $\sim 10\times$ the fine-tuned rows. Its structure is right (the surfaces of a frame land in the right places relative to each other) and its distances are not: it predicts a 48° field of view where the camera has 62°, and even after removing all scale per frame its AbsRel is 0.24–0.54 against 0.04–0.13 for DA-v2 Metric on the same frames. The failure is specific to this input—VGA (640 $\times$ 480), close indoor range, motion blur—and we verified the wrapper against a synthetic room with exact ground truth (F@5 cm-level agreement, AbsRel 0.008 after one scale), so it is the model on this data, not the plumbing. The practical reading for

a product team: a model that takes intrinsics is not a substitute for depth labels from the target device.

C. Context, Exp. 1: the price of each depth source

What the table says directly:

- 1) **The pipeline loses ~2 cm; LiDAR adds ~1 cm.** The gap `lidar-gt` is 0.98 ± 0.25 cm (0.66–1.33), the same in every room including the bathroom; $F@5$ cm 0.93 vs 0.97. The rows differ clearly only at $\tau=2$ cm (0.54 vs 0.86): iPad LiDAR is wrong at the 2–4 cm level, not 5 cm.
- 2) **Monocular depth with the right scale on every frame (oracle) is 2.29 ± 1.11 cm behind LiDAR (0.9–3.8).** Bedroom 47333774 (3.8) and the bathroom (3.4) are farthest; the development kitchen 42444474 (0.9) and kitchen 45261556 (1.6) closest—more room-dependent than LiDAR’s std suggests (§VI-D).
- 3) **Two hundred sparse metric points per frame recover the oracle.** `sparse_points` lands at 6.0 cm, 0.6 cm from the oracle and $3.8\times$ better than one-off calibration, with the same per-scene ordering as the oracle (Table VIII). The remaining gap to LiDAR (2.9 cm) is shape, not scale.
- 4) **One calibration per scene (`per_scene`) is $4.5 \pm 1.1\times$ worse than the oracle (3.1–6.4 $\times$), and—importantly—the absolute values cluster at 20–26 cm in every room** (std 2.1 cm, lower than the oracle’s): not one room failing, but a property of affine-invariant models with one scale per scene (§VI-B).
- 5) **The metric model is worst (54.3 cm, F 0.03) and most variable (42–71 cm); accuracy 70.3 cm vs completeness 38.3 cm: the surfaces are all there but in the wrong place (§VI-C).** Kitchen 45261556, where mono+oracle is second best (4.5), is where the metric model is worst (71.0)—right shape, wrong scale. This is the baseline of §V-A: fine-tuning this same model on ARKitScenes with a LiDAR teacher removes most of that error.

D. Context, Exp. 2: voxel size (GT depth)

Going from 2 to 4 cm costs 0.9 cm Chamfer, mostly completeness (1.3→2.8: coarser corners and edges), not accuracy (1.05→1.29). From 4 to 8 cm costs another 0.7 cm and $F@5$ cm drops 0.97→0.93—at 8 cm discretisation exceeds LiDAR’s own error. Fusion takes 1–3 s per scan at every size (CPU, Open3D), so voxel size does not drive cost; inference does (§VII). **4 cm is used throughout:** its discretisation (2.1 cm) is smaller than the depth-source effects being measured (LiDAR +1, mono +4, `per_scene` +20) and the mesh stays small enough for a phone.

E. Context, Exp. 3: frame stride—how many frames per second to capture

Faro provides frames at ~ 4.1 fps (2,352 frames / 577 s); strides 1/5/10/20 therefore correspond to $\approx 4 / 0.8 / 0.4 / 0.2$ fps (391 / 78 / 40 / 20 frames per scan on average). Three rows per stride separate the causes: GT control (coverage

only), mono+oracle (+ model shape), mono+`per_scene` (+ scale).

- **Control:** GT accuracy is constant at 1.2–1.3 cm at every stride; the growing Chamfer (2.1→16.8 cm) is pure completeness. On this dataset stride measures **coverage**: a room swept once develops holes below 1 fps.
- **mono+oracle tracks the control plus a constant 2.7–3.3 cm**, and its accuracy *improves* with stride (5.9→3.4): fewer frames means fewer overlapping surfaces from different views; 2D AbsRel is 0.032 at every stride, confirming the model did not change.
- **mono+`per_scene` is flat at ~23 cm up to stride 10:** at stride 5 it saves $5\times$ the time (345→70 s) at the same Chamfer (22.9 vs. 22.5), because the error is already dominated by scale. At stride 20 `per_scene` fits on 10 of 20 frames (nearly a half-scene oracle), so that cell should not be over-interpreted.
- **For the business:** with good depth (LiDAR) capture ≥ 1 fps for a complete room ($F@5$ cm > 0.9); with mono+`per_scene`, 0.8 fps gives the same result as 4 fps, $5\times$ faster—quality is capped by scale, not by frame count.

At stride 20 every row has the same holes (areas the camera passed quickly): holes are a property of the scan, not of the depth source.

F. Context, Exp. 4: model size (oracle scale)

L→S: $13\times$ smaller, $6\times$ faster, +1.0 cm (F 0.79→0.72); every scene moves the same way except the bathroom, where Small is slightly *better* (7.5→7.1). S→MiDaS: similar parameter count, a further $3\times$ faster, but $2.1\times$ the Chamfer and F 0.41—a matter of model generation (DA-v2 was trained with 62M unlabelled images), not size. At oracle scale DA-v2 S (6.35 cm) is still 3.3 cm behind LiDAR (3.0). Model size is not the main bottleneck: Table VII shows scale (oracle→`per_scene`, +17.6 cm) costs $17\times$ more than model size (L→S, +1.0 cm).

G. Context, Exp. 5: does LiDAR confidence help?

ARKit ships a $\{0, 1, 2\}$ confidence map with every LiDAR frame. Masking or weighting by it is the cheapest possible improvement to the “iPhone Pro today” row. Nothing moves: dropping confidence-0 pixels or weighting by tier changes Chamfer by ≤ 0.03 cm; keeping only confidence-2 pixels improves accuracy slightly (2.23→2.08) but loses completeness (3.85→5.23, worst in the closet-heavy bedroom 47429736: 3.1→5.8). The 1 cm LiDAR–laser gap is not a low-confidence-pixel problem; it is the sensor’s resolution and its 2–4 cm noise everywhere.

H. Numbers carried into §VI–VII

VI. DISCUSSION

Material: per-frame scale analysis on 6 scenes (`analyze_scale_drift.py`, stride 5, 471 frames), the failure-frame study of scene 42444474, and Tables II–VIII. Per-frame AbsRel uses protocol 2.

TABLE VII
SAME PIPELINE, 4 CM VOXELS, EVERY FARO FRAME; ONLY `DEPTH.SOURCE / DEPTH.ALIGNER` CHANGE. `SPARSE = PER-FRAME FIT ON 200`
LiDAR-SAMPLED POINTS (VIO PROXY; UPPER BOUND OF REAL VIO).

Depth source	Aligner	Chamfer	Acc.	Compl.	F@2	F@5	F@10	AbsRel (2D)	Time/scan (s)
Faro laser (gt)	identity	2.1 ± 0.5	1.3	2.8	0.86	0.97	0.99	0	1.6
ARKit LiDAR (lidar)	identity	3.0 ± 0.6	2.2	3.9	0.54	0.93	0.98	0.018	1.7
DA-v2 L (mono)	oracle_frame	5.3 ± 1.3	5.9	4.7	0.46	0.79	0.90	0.032	366
DA-v2 L (mono)	sparse_points	6.0 ± 1.7	6.4	5.5	0.38	0.74	0.88	0.036	360
DA-v2 L (mono)	per_scene	22.9 ± 2.1	24.2	21.6	0.08	0.23	0.41	0.237	351
DA-v2 Metric-Indoor	identity	54.3 ± 10.6	70.3	38.3	0.01	0.03	0.06	0.382	352

TABLE VIII
CHAMFER (CM) PER SCENE; COLUMN ORDER AS IN TABLE I. 42444474 IS THE DEVELOPMENT SCENE.

Run	42444474 kitchen/glass	47333774 bedroom	47115299 living	42897743 bathroom	47429736 bedroom+closets	45261556 kitchen	mean
gt	1.8	1.2	1.9	2.8	2.4	2.3	2.1
lidar	2.8	2.4	2.9	4.1	3.1	3.0	3.0
mono + oracle	3.8	6.2	4.8	7.5	5.2	4.5	5.3
mono + sparse	4.5	6.1	5.5	9.1	5.9	4.6	6.0
mono + per_scene	24.1	26.0	19.9	23.2	21.4	23.1	22.9
mono metric	48.5	61.6	41.5	48.9	54.1	71.0	54.3

TABLE IX
`DEPTH.SOURCE : GT, SDF_TRUNC = 3 × VOXEL.`

Voxel	Chamfer	Acc.	Compl.	F@2	F@5	F@10	Fusion (s)
2 cm	1.18 ± 0.19	1.05	1.31	0.92	0.99	1.00	2.5
4 cm	2.06 ± 0.54	1.29	2.83	0.86	0.97	0.99	1.5
8 cm	2.76 ± 0.85	1.74	3.79	0.76	0.93	0.97	1.3

A. Why the LiDAR-teacher fine-tune works—and why it is not enough

We ran the per-frame analysis of §VI-C on R3 as well (every fifth frame of the six test rooms, 471 frames): per frame, the GT/pred ratio (median over the frame) and AbsRel under different amounts of scale help (Table XIII).

- 1) **Fine-tuning works because it learns the domain’s multiplier.** R3 with no calibration (AbsRel 0.134) is on par with the pretrained model given a per-room scale fitted on ground truth (0.127): what a one-off calibration per room provides has been moved into the head’s weights, using LiDAR labels from other rooms. With the encoder frozen, shape barely changes (oracle 0.035 → 0.042); the whole gain is scale.
- 2) **It is not enough because the per-frame swing remains.** Within one room, frames at p5 and p95 still need factors $\sim 1.5\times$ apart, as before fine-tuning, and that swing is nearly all of what is left: with the right scale on every frame R3’s AbsRel drops from 0.134 to 0.051 (90 % of the way to the oracle), whereas one (S, T) per room only reaches 0.117. The failing frames are those of §VI-B–VI-C (blank walls, white cabinets, close downward views with no size cue). The missing information is not in a single image and does not come from a better teacher (the laser teacher does no better, §V-B); it has to come from metric data at run time.

- 3) **Fine-tuning and per-frame metric points are complementary, not alternatives.** Fine-tuning removes the domain bias with nothing at run time (54.3 cm → 15.7 cm); per-frame sparse points remove the per-frame swing (relative model: 22.9 cm → 6.0 cm, §V-C). The 2D bound for combining them is the 0.051 “right scale on every frame” row above; measuring R3 + `sparse_points` in 3D is the most direct next step (§VIII).

B. Why per_scene loses to the oracle by 4.5×: the model’s scale follows what is in the frame, not time

An affine-invariant model promises disparity $\approx a \cdot (1/\text{depth}) + b$ with (a, b) depending on the image. If (a, b) were constant over a scan, one calibration (`per_scene`) would equal the oracle. We measured how constant it is by refitting (s, t) on every fifth frame (Fig. 5, Table XIV). Three consequences:

- 1) **No number of fit frames helps per_scene:** the (S, T) fitted on 10 spread frames already lands within $\pm 13\%$ of the median oracle scale in five of the six scenes (0.87–1.17×; the exception, 47333774, is 0.67×). The problem is that the median does not apply to most frames, not that it is estimated badly.
- 2) **A 7.5× per-frame error becomes a 4.5× 3D error not because TSDF averages it out but because the error concentrates in a minority of frames:** frames with `per_scene` AbsRel > 1 (placed $\geq 2\times$ wrong) are downward shots of table-tops or floor at 0.5–0.8 m and frame-filling blank walls or cabinets. These frames carry no size cue, so the model guesses disparity at “room scale”; multiplied by the room’s (S, T) , a table at 0.5 m lands at 1.5–2 m as a phantom surface in the middle of the room—the red blob in every `per_scene` panel of Fig. 4.

TABLE X
CHAMFER / ACCURACY / COMPLETENESS (CM) AND F@5 CM VS. STRIDE.

Stride ($\approx$ fps)	Frames	GT: Ch / acc / comp	F	mono oracle: Ch / acc / comp	F	mono per_scene: Ch / acc / comp	F	mono time (s)
1 (4)	391	2.1/ 1.3 / 2.8	0.97	5.3/ 5.9/ 4.7	0.79	22.9/ 24.2/ 21.6	0.23	345
5 (0.8)	78	4.4/ 1.2 / 7.5	0.91	7.3/ 4.3/ 10.3	0.77	22.5/ 19.6/ 25.4	0.24	70
10 (0.4)	40	8.9/ 1.2 / 16.6	0.83	11.6/ 3.6/ 19.5	0.70	25.7/ 20.3/ 31.0	0.21	36
20 (0.2)	20	16.8/ 1.2 / 32.3	0.68	19.7/ 3.4/ 36.1	0.56	33.8/ 21.0/ 46.6	0.15	19

TABLE XI
ORACLE_FRAME FOR EVERY MODEL, 4 CM, EVERY FRAME—PURE “SHAPE,” EXCLUDING THE SCALE PROBLEM OF §V-C.

Model	Params	Chamfer	F@5	AbsRel	δ_1	ms/frame (MPS)	s/scan
DA-v2 Large	335M	5.33 $\pm$ 1.33	0.79	0.032	0.986	871	348
DA-v2 Small	25M	6.35 $\pm$ 1.37	0.72	0.042	0.979	121	55
MiDaS v2.1 small	21M	13.24 $\pm$ 3.26	0.41	0.095	0.904	27	16

TABLE XII
LiDAR ROW WITH CONFIDENCE MASKING / INTEGER TSDF WEIGHTS (4 CM, EVERY FRAME).

Variant	Chamfer	Acc.	Compl.	F@5
all pixels, weight 1 (Exp. 1)	3.04 $\pm$ 0.57	2.23	3.85	0.93
mask confidence ≥ 1	3.07	2.17	3.98	0.93
mask confidence = 2	3.66	2.08	5.23	0.93
weights [0, 1, 2]	3.07	2.17	3.98	0.93
weights [1, 2, 4]	3.04	2.23	3.85	0.93

3) **The effect is general, not a property of the development scene:** 47429736 (bedroom+closets) has close-ups in 53/101 frames yet per_scene AbsRel 0.197, the same as a scene with 3/93 (45261556: 0.172); and the five unseen scenes give per_scene Chamfer 19.9–26.0 cm, bracketing 42444474 (24.1).

System implication. The assumption “one scale per scene” is false for this model family on real room video. Two ways remain: (a) *per-frame* metric data—3D feature points from VIO, which ARKit/ARCore deliver with the pose anyway (sparse_points); or (b) drop or down-weight cue-less frames before fusion—cheaper, but removing close-ups still leaves 4 $\times$ the oracle, so it can only supplement (a). Exp. 1 confirms (a): 200 points per frame bring the row to 6.0 cm, 0.6 cm from the oracle, with the same per-scene ordering. The per-frame fit needs no history and tolerates frames without points by carrying the previous (s, t), exactly what a tracker outage looks like.

C. Why the “metric” model gives 54.3 cm: right on average, wrong per frame

DA-v2 Metric-Indoor should output metres directly. Per-frame analysis over the six scenes: In 3D the same surface from several frames is placed at several distances and TSDF averages them into a thick slab: accuracy 70.3 cm but completeness 38.3 cm (Table VII)—everything is there, in the wrong place. The worst scene (kitchen 45261556, 71.0 cm) is among the model’s best shapes under the oracle (4.5 cm), confirming a pure scale effect. The worst frames in every

Question	Number
R3 vs. pretr. (3D, 6 rooms)	54.3 cm $\rightarrow$ 15.7 cm, F@5 cm 0.03 $\rightarrow$ 0.24, every room better
R3 vs. pretr. (2D, 6 rooms)	AbsRel 0.382 $\rightarrow$ 0.134, δ_1 0.277 $\rightarrow$ 0.812
R3 vs. pretr. (2D, 20 rooms)	AbsRel 0.410 $\rightarrow$ 0.130, better in 19/20
Faro vs. LiDAR teacher	14.6 cm / 16.6 cm / 15.7 cm (R1/R2/R3): not measurably different
pipeline loss at 4 cm	2.1 $\pm$ 0.5 cm
iPad LiDAR vs laser	+1.0 $\pm$ 0.3 cm, F@5 cm 0.93 (confidence weighting: no change)
mono ceiling (scale known per frame)	+2.29 $\pm$ 1.11 cm from LiDAR, F@5 cm 0.79
mono with 200 sparse points/frame	6.0 cm, F@5 cm 0.74 (0.6 cm from oracle)
price of “one scale per scene”	4.5 $\times$ oracle $\rightarrow$ 22.9 cm, F@5 cm 0.23
intrinsic-conditioned model (Depth Pro)	155.4 cm, F@5 cm 0.00 — intrinsic do not replace in-domain labels
voxel 2 $\rightarrow$ 4 $\rightarrow$ 8 cm	1.2 $\rightarrow$ 2.1 $\rightarrow$ 2.8 cm
fps needed (LiDAR/GT)	≥ 0.8 fps for F@5 cm ≥ 0.9 ; 0.4 fps leaves 0.83
L $\rightarrow$ S $\rightarrow$ MiDaS	5.33 $\rightarrow$ 6.35 $\rightarrow$ 13.24 cm at 871 $\rightarrow$ 121 $\rightarrow$ 27 ms/frame

scene look alike: a single flat, uniformly coloured surface filling the frame (white cabinets, tiles) with nothing distant to compare against—the model’s prior pushes flat surfaces away.

Implication: a metric model does not remove calibration; it reduces it from two parameters per frame (s, t) to **one** (s), which is a real advantage: a handful of VIO points suffice to fit one scalar stably, whereas (s, t) needs points spread over the depth range. The other way to get that scalar is to put it in the weights: fine-tuning the same checkpoint with LiDAR labels from real iPad frames (R3, 19 896 frames) moves the row to 15.7 cm with no run-time fit at all (§V-A), the deployable answer for a device that has neither LiDAR nor reliable VIO points; §VI-A shows that what it fixes is the domain bias, while the per-frame swing remains.

D. Which rooms are hard, and for whom

Ranking scenes by Chamfer per row (Table VIII): Faro/LiDAR go from bedroom 47333774 (easiest) to the bathroom (1.2–2.8 / 2.4–4.1); mono+oracle from the development kitchen 42444474 and kitchen 45261556 to the bathroom

TABLE XIII
PER-FRAME SCALE OF THE PRETRAINED METRIC MODEL VS. THE LiDAR-TEACHER FINE-TUNE R3, SIX TEST ROOMS, EVERY FIFTH FRAME. ABSREL USES PROTOCOL 2.

	Pretrained	R3	Meaning
median(GT / pred) per room	0.69–0.85	0.93–1.21	the domain bias is gone: pretrained is 15–31% too far in every room; R3 is within $\sim 20\%$ and not always in the same direction
per-frame swing within a room (p95/p5 of GT/pred)	$1.33\text{--}1.79\times$ (mean 1.55)	$1.26\text{--}1.92\times$ (mean 1.52)	fine-tuning does not reduce the per-frame swing
AbsRel, raw	0.381	0.134	pretrained + per-room GT calibration $\approx$ R3 with no calibration at all
AbsRel with one (S, T) per room fitted on 10 GT frames	0.127	0.117	
AbsRel with the right scale on every frame (one factor)	0.045	0.051	the per-frame scale is almost all of R3’s remaining error
AbsRel, oracle (s, t) per frame	0.035	0.042	shape hardly changes (frozen encoder)

TABLE XIV
DA-V2 LARGE, PER-SCENE VALUES ACROSS THE SIX SCENES.

	Range over scenes	Meaning
s p95 / p5 within one scene	2.6–4.9 $\times$	the central 90% of frames need scales $3\text{--}5\times$ apart even the middle half swings $\pm 25\%$
s p75 / p25	1.47–1.56	
corr(s , time)	$-0.33 \dots +0.30$ (sign flips)	not cumulative drift: no direction
corr(s , frame’s median depth)	$-0.27 \dots -0.75$ (negative in every scene)	close-up frames need a different s : view-dependent
AbsRel oracle $\rightarrow$ per_scene	$0.032 \rightarrow 0.239$ (7.5 $\times$)	in 2D, before fusion, one scale costs $7\times$ close-ups hurt but are not the only cause a disparity shift is needed in every scene
per_scene without close-ups (<1 m)	0.152 (still $5\times$ oracle)	
scale-only per frame, depth space	0.331 (worse than per_scene)	

(3.8–7.5); mono+per_scene from living room 47115299 to bedroom 47333774 (19.9–26.0).

- **The bathroom is hard for every source** (LiDAR +1.3 from Faro, oracle +3.4 from LiDAR): the mirror yields phantom depth for LiDAR and model alike, glossy tiles reflect; it is the only scene where the oracle fit produced $s \leq 0$ on some frames (a frame-filling mirror). If the target market is bathroom renovation, this paper’s averages are optimistic.
- **Standard bedroom 47333774 is easiest for LiDAR but second-hardest for mono** (oracle 6.2): a small room scanned close to walls and bed leaves most frames at 1–2 m of blank wall with no cue. Rooms with “lots of stuff” (kitchen 45261556: 10 cabinets; living room: 4 tables, 2 sofas) are easy for mono—the model likes objects of known size.
- The mono difficulty order does not match LiDAR’s $\Rightarrow$ **“hard scenes” must be assessed per source**; LiDAR experience does not predict where mono fails.

E. Visible only in 3D: mono disagrees with itself across views

Exp. 3 shows what 2D metrics cannot: reducing frames from 391 to 20 (stride 20) **improves mono+oracle accuracy from 5.9 to 3.4 cm** while 2D AbsRel stays at 0.032–0.033 and GT accuracy at 1.3–1.2 cm. Even with every frame perfectly scaled, the shape of the same surface from different views **disagrees** at the ~ 2 cm level, and fusing more frames thickens the surface rather than sharpening it—the multi-view consistency LiDAR has and a single-image model lacks. This is the part of the oracle $\rightarrow$ LiDAR gap (2.29 cm) that scale alignment cannot fix, and the reason systems like SimpleRecon use multi-image cost volumes (§II-B). The sparse row inherits it: its remaining gap to LiDAR (2.9 cm) is this shape term.

F. Conclusions for system design

VII. BUSINESS IMPLICATIONS

Numbers from §V; times are Apple Silicon / MPS and serve only for ratios (§VIII).

	Range over scenes	Meaning
median(GT / pred) per scene	0.69–0.85	the model says 15–31% too far on average (per-scene bias)
GT / pred within a scene	e.g. 0.41–0.86, 0.43–1.35	frames in one room are wrong from –59% to +35%
oracle t in inverse space	–0.11 ... – 0.02 (≈ 0)	no shift : the shape is right in the affine sense
AbsRel raw $\rightarrow$ scale-only/frame $\rightarrow$ oracle (s, t)	0.381 $\rightarrow$ 0.045 $\rightarrow$ 0.035	one scalar per frame recovers 97% of the gap
AbsRel with one (S, T) per scene	0.127	like per_scene for the relative model: the median is not enough—and what fine-tuning moves into the weights (§VI-A)

Assumed beforehand	What the data say
mono’s problem is coarse shape	shape is 2.29 cm behind LiDAR; scale is 18 cm behind
one calibration per room suffices	no: 22.9 cm in every room; scale is view-dependent
a metric model removes the scale problem	no: but it reduces it to one parameter per frame
a metric model cannot be fixed without a laser scanner	largely can: a phone-LiDAR teacher takes it from 54.3 cm to 15.7 cm in every room (§V-A); the rest is per-frame scale (§VI-A)
a few sparse metric points cannot fix a dense map	they can: 200 points/frame $\rightarrow$ within 0.6 cm of the oracle
a bigger model means a more accurate room	+1.0 cm S $\rightarrow$ L vs. +17.6 cm from scale
more frames mean more accuracy	more frames mean more coverage; mono accuracy improves with fewer frames
LiDAR confidence maps are free accuracy	no measurable gain from masking or weighting (Exp. 5)
LiDAR’s hard scenes are mono’s hard scenes	no (blank bedroom: hard for mono, easy for LiDAR)

A. *The one number a decision-maker needs: “how crooked is the room” per use case*

Table XV translates Table VII into product language, requiring that a use case have $\geq 90\%$ of the surface within τ of the truth (recall@ τ).

- **iPad Pro today is enough for “furniture placement” but not for “renovation measurement”**: at 2 cm LiDAR reaches only half, and even Faro through this pipeline reaches 0.84 (the 4 cm voxel eats it; §V-D shows 2 cm voxels reach 0.92). A product promising centimetre accuracy needs finer voxels plus multiple LiDAR passes, or a laser. Confidence weighting does not move this (Exp. 5).
- **Devices without LiDAR serve no use case** with one-off calibration (per_scene 22.9 cm) or a raw metric model (54.3 cm). Fine-tuning that metric model with a teacher a phone collects itself (R3) moves it a large step toward the “rough overview” tier without any run-time calibration or sensor (15.7 cm, recall@10 cm 0.51 vs. 0.10 pretrained, §V-A), but still short of its 0.9 threshold and far from “furniture placement”—the remainder is per-frame scale (§VI-A).
- **With per-frame sparse points the same device reaches 6.0 cm, recall@5 cm 0.77**—the “furniture placement” tier at a level close to iPad Pro, and the mono ceiling (oracle 5.3 cm) is only 2.29 cm behind LiDAR. This is the gap to invest in, not the model.

B. *Where to invest: scale first, model second*

The order the data dictate: (1) **solve per-frame scale $\rightarrow$ (2) ship DA-v2 S on device by default $\rightarrow$ (3) consider L or a compressed model once (1) is stable**. Investing in the model

before scale returns <3 cm on a 22 cm base—invisible in the product.

C. *The LiDAR teacher is a data advantage that needs no laser scanner*

The main result (§V-A) translates directly into strategy: **an app with iPhone/iPad Pro users can collect its own teacher during normal use** (RGB, LiDAR depth and pose arrive together from ARKit) and use it to train the model that runs on devices without the sensor.

- Data cost: one to two minutes of video per room from devices already in the field, versus a \$20k laser scanner, an operator and cross-sensor registration.
- Measured effect on 6 + 20 unseen rooms: every test room improves (54.3 cm $\rightarrow$ 15.7 cm); AbsRel 0.410 $\rightarrow$ 0.130 on the Validation fold.
- A laser teacher is not measurably better (§V-B), so there is no accuracy case for laser capture for this purpose.
- Caveats: only the iPad camera is measured—a model for Android needs data from many cameras (§VIII); and going from 10 to 24 rooms did not measurably help (§V-B), so the room count at which more data pays off is unknown.

D. *Cost per scan*

Per room (~ 95 s of video ≈ 391 frames at 4 fps; MPS, unoptimised PyTorch): At 4 fps DA-v2 L takes $3.6\times$ the video length—not real time even on a laptop; DA-v2 S is at $0.5\times$ (processable right after capture). Exp. 3 says per_scene quality needs only 0.8 fps (same Chamfer), bringing L to 68 s and S to 9 s per room; with per-frame scale, ≥ 0.8 –4 fps is needed for coverage (F@5 cm 0.77 at 0.8 fps vs 0.79 at 4 fps—0.8 fps

TABLE XV
RECALL@ τ AVERAGED OVER 6 SCENES. $\checkmark \geq 0.9$, ~ 0.75 – 0.9 , $\times < 0.75$.

Use case	τ	Faro	iPad LiDAR	mono + oracle	mono + sparse	mono + per_scene	mono ft R3 (LiDAR)	mono metric
room overview / virtual walk-through	10 cm	$\checkmark$ 0.98	$\checkmark$ 0.97	$\checkmark$ 0.93	$\checkmark$ 0.91	$\times$ 0.47	$\times$ 0.51	$\times$ 0.10
furniture placement / space planning	5 cm	$\checkmark$ 0.96	$\checkmark$ 0.91	$\sim$ 0.82	$\sim$ 0.77	$\times$ 0.27	$\times$ 0.28	$\times$ 0.04
floor-area sale / renovation (walls, trim)	2 cm	$\sim$ 0.84	$\times$ 0.52	$\times$ 0.47	$\times$ 0.39	$\times$ 0.09	$\times$ 0.09	$\times$ 0.01

TABLE XVI
WHAT EATS MONO’S ERROR, SIX SCENES.

Error source	Size (Chamfer)	Remedy	Cost
one scale per scene $\rightarrow$ per frame	– 18 cm (22.9 $\rightarrow$ 5.3)	per-frame metric points from VIO (free from ARCore/ARKit)	one aligner (sparse_points); measured: 22.9 $\rightarrow$ 6.0
teach the model the scale instead	– 38.6 cm (54.3 $\rightarrow$ 15.7)	fine-tune on rooms captured with a phone’s own LiDAR (R3)	one training run ($\sim$ 2 h on an RTX 3070); no run-time cost
model S $\rightarrow$ L	–1.0 cm (6.35 $\rightarrow$ 5.33)	13 $\times$ larger model, 6 $\times$ slower	on-device / cloud compute
voxel 4 $\rightarrow$ 2 cm	–0.9 cm	$\sim$ 4 $\times$ larger mesh	memory / bandwidth
MiDaS $\rightarrow$ DA-v2 S	–5.7 cm	newer model, same size	zero
LiDAR confidence weighting	$\pm$ 0.03 cm	—	(not worth it)

Stage	LiDAR	DA-v2 L	DA-v2 S	MiDaS s
depth per frame	0 (sensor)	871 ms	121 ms	27 ms
depth per scan @ 4 fps	0	341 s	47 s	11 s
depth per scan @ 0.8 fps (stride 5)	0	68 s	9 s	2 s
align + fusion + extract	1.6 s	$\sim$ 9 s	$\sim$ 6 s	$\sim$ 4 s

is still the sweet spot). Fusion and extraction are a rounding error ($< 3\%$ of L’s time); TSDF is not what to optimise. Data: at 0.8 fps a room is ~ 80 images at 640×480 (≈ 10 – 20 MB JPEG) plus poses and sparse points—a few seconds to upload, so “capture on device, process on a server with L” is feasible without waiting for on-device models. The `roomscan_web` package (§III) is that architecture in its minimal form: upload a capture, poll a job, fetch the mesh.

E. Positioning

What can be said from these data: “*The same system gives 3 cm on a LiDAR device and about 6.0 cm on a device without one, provided the tracker’s sparse points are used per frame—what separates the two numbers is measurable engineering, not model research.*” What **cannot** be said: “no LiDAR, same accuracy”—the data say the opposite in every scene.

F. Risks the numbers do not cover

VIO pose on low-end Android may be worse than ARKit’s, adding error to every row equally (not measured); regions Faro did not see (behind cabinets, high ceilings) are outside the numbers, so a product must handle “the part not captured” itself; the bathroom is the worst scene for every source that gets its scale right (Faro 2.8, LiDAR 4.1, oracle 7.5, sparse 9.1)—if the target market is bathroom/kitchen renovation, more such scenes must be collected; and the sparse-point row uses LiDAR-sampled points, cleaner than real VIO features (§VIII).

VIII. LIMITATIONS AND FUTURE WORK

A. Limitations of the measurement

(a) **The reference is not an independent ground truth.** It is fused from Faro depth with the same pipeline (1 cm voxels) and uses the **same poses** as every row, so (i) the gt row is non-zero and measures our own discretisation, (ii) VIO pose drift is removed from every number—a real phone system incurs additional pose error this paper does not measure, and (iii) areas Faro did not see are masked from every source equally, so “completeness” means “complete relative to what Faro saw,” not the whole room.

(b) **Evaluated at ~ 4 fps only.** Faro depth exists only for the frames Apple kept, so every source is evaluated on that set although LiDAR runs at 60 fps and RGB at 30 fps. What a real pipeline gains or loses from denser frames (smoothing vs. accumulated error) is not measured; Exp. 3 answers coverage, and compute-vs-accuracy only partly.

(c) **Fusion resolution** 256×192 , chosen to be fair to LiDAR, downsamples the monocular model (which predicts at 640×480) before fusion; DA-v2’s sharp-edge advantage does not appear in the 3D numbers. “Mono at its best” would require fusing at higher resolution and accepting a disadvantaged LiDAR row.

(d) **Six rooms (3D) / 26 rooms (2D), one dataset, one device.** Every mesh number comes from the six rooms that have a reference mesh; Exp. 7 extends the 2D measurement of the main result to $6 + 20$ rooms but says nothing about mesh quality there, and uses every second Faro frame. Every scan is an iPad Pro in homes Apple selected; offices, empty

	RoomPlan / Polycam (LiDAR)	this work on a LiDAR device	this work without LiDAR, one-off calibration	this work without LiDAR + VIO points
market measured accuracy	iPhone/iPad Pro not disclosed	same 3.0 cm, F@5 cm 0.93	Android + regular iPhone 22.9 cm, F@5 cm 0.23 (15.7 cm R3 fine-tuned, no calibration)	same 6.0 cm, F@5 cm 0.74 (VIO-proxy upper bound)
use case served	(claims floor plan)	furniture placement	not even overview	furniture placement (nearly)
needs from device	LiDAR	LiDAR + VIO	camera + VIO	camera + VIO + feature points

rooms and low light are absent, and 42444474 was used during development, so its numbers are in-sample—per-scene tables show whether the other five confirm.

(e) **The fine-tuned rows are selected on three validation scenes.** Exp. 6 keeps the checkpoint with the lowest AbsRel on the 3 validation rooms; that ranking ($R3 > R2 > R1$) does not reproduce on the six test rooms ($R1 \geq R3 \geq R2$ in Chamfer). Three rooms are too few to separate checkpoints that differ by ~ 0.01 AbsRel, so the Exp. 6 ordering between teachers should be read as “no measurable difference”, and a larger validation set is the first thing to add. The six test rooms were never used for training, validation or checkpoint choice. Moreover R1 and R2 do not differ in the label alone: R2 sees $\sim 2.4\times$ more distinct views (8656 vs. 3594 frames at equal steps, §IV-E); a controlled teacher comparison needs R2 trained on R1’s Faro timestamps, which we have not run. This does not affect the main question (R3 vs. pretrained), where both rows are scored on the same test frames.

(f) **The no-fine-tuning baseline is one model, one implementation, one input resolution.** Depth Pro is run through its HuggingFace implementation on 640×480 frames upsampled to 1536^2 by its own processor, with the true focal length supplied; we did not test Apple’s reference code, higher-resolution input (the dataset’s 1920×1440 frames were not downloaded), or the other intrinsics-conditioned models (UniDepth, Metric3D). The claim we make is about this model on this input, not about the family.

(g) **One fine-tuning recipe, one architecture, one label preparation.** The LiDAR label (256×192) is up-sampled with nearest-neighbour to the colour grid (640×480 , then 518×686) before the loss, rather than down-sampling the prediction to LiDAR’s own grid, and confidence ≥ 1 (not $= 2$) is used; neither choice has been ablated (R2 landing close to R1, whose Faro labels are high-resolution, suggests the effect is small but does not prove it). Only DA-v2 Metric-Indoor Large with a frozen encoder was fine-tuned; whether unfreezing the encoder, a different loss or another architecture would widen or close the Faro-vs-LiDAR teacher gap is untested, as is training on more than 24 rooms.

(h) **Times are Apple Silicon (MPS) with unoptimised PyTorch**, not representative of on-device inference (CoreML / NNAPI); use them for *ratios* between models, not absolute values.

B. Limitations of the system

(a) **per_scene still calibrates on GT:** it is the upper bound of one-off calibration; a real system using a measured wall or camera height would do worse. §VI-B shows even this bound is far from the oracle because scale swings per view—the assumption, not the calibration method, is the problem.

(b) **sparse_points is evaluated on a proxy.** ARKitScenes ships no ARKit feature points, so the aligner sees 200 pixels sampled from the LiDAR frame at high confidence. Real VIO points are triangulated, noisier, and cluster on texture; the row is therefore an *upper bound* of a VIO-based system. The loader exposes a noise knob (`sparse_noise`) and point count for sensitivity studies; an evaluation on real ARKit/ARCore point clouds needs our own captures (`dataio/custom.py` reads them).

(c) **No outlier rejection before fusion:** every posed frame is integrated equally. §VI-B shows a minority of frames (close-ups, blank walls) create most of `per_scene`’s error; frame weighting is what Exp. 5’s mechanism provides, applied here only to LiDAR confidence so the main tables measure the depth source alone.

(d) **Poses come from the dataset:** there is no tracking in the system; an MVP uses ARKit/ARCore poses of similar quality to the trajectories used here, possibly worse on low-end Android.

(e) **No GPU fusion:** fusion is < 2 s per scan on CPU, so it is not a bottleneck; a GPU backend fits behind the same TSDFusion interface if higher resolutions are pursued.

C. Future work (ordered by impact on the MVP decision)

- 1) **R3 + per-frame scale (sparse_points on the fine-tuned model):** §VI-A shows that almost all of R3’s remaining error is a per-frame factor (AbsRel 0.134 $\rightarrow$ 0.051 with the right scale); one new Exp. 6 row answers how close “LiDAR teacher + VIO points” gets to a sensor. Together with a *controlled teacher comparison* (R2 on R1’s Faro timestamps) and a *larger validation set*.
- 2) **Training data from more phone cameras:** R3 has only seen the iPad camera; it must be re-measured on iPhone / Android cameras before claiming it serves devices without LiDAR in general.
- 3) **Real VIO points:** capture rooms with the app (ARKit `rawFeaturePoints` / ARCore point cloud) plus a reference (tape measurements or a rented laser scanner)

and rerun the `sparse_points` row on real, noisy, textured points; sweep `sparse_noise` and point count on ARKitScenes meanwhile.

- 4) **Metric model + scale-only per frame:** §VI-C suggests one parameter per frame suffices; test whether one VIO point per frame is enough.
- 5) **Frame selection / weighting** for mono: drop or down-weight frames with median depth < 1 m or low disparity variance before fusion, using the per-pixel weight channel already used in Exp. 5.
- 6) **Android capture (ARCore depth source):** `depth_sources/arcore.py` has its registry slot; data must be collected with a reference.
- 7) **Compressed models:** Exp. 4 provides the L / S / MiDaS baseline; a distilled or quantised model plugs in as `models/<x>.py` and reruns Exp. 4.
- 8) **Fusion above 256×192 for mono,** to measure what downsampling costs DA-v2.
- 9) **Semantic layer** (wall / floor / furniture labels on the mesh) for usable-area computation; sits after post-processing and does not affect the geometry numbers.

IX. CONCLUSION

We asked whether transfer learning with depth from a phone’s own LiDAR really makes a metric monocular model better than its pretrained self, measured against the laser scanner that ARKitScenes provides as ground truth, and answered it through a room-reconstruction pipeline (per-frame depth $\rightarrow$ scale alignment $\rightarrow$ TSDF $\rightarrow$ mesh) in which **the depth source is the single variable switched from config**. The numbers say:

- 1) **Yes, in every room we can measure.** Fine-tuning only the head of DA-v2 Metric-Indoor with iPad LiDAR labels on 24 rooms (R3) lowers Chamfer from 54.3 cm to 15.7 cm on all six held-out rooms ($F@5$ cm $0.03 \rightarrow 0.24$; AbsRel against Faro $0.382 \rightarrow 0.134$), and per-frame AbsRel from 0.410 to 0.130 on 20 further Validation-fold rooms (better in 19/20)—with no calibration and no sensor at run time.
- 2) **The teacher does not need to be a laser scanner.** Faro, LiDAR on the same rooms and LiDAR on more rooms give 14.6 cm / 16.6 cm / 15.7 cm, within the across-room spread, so the training data for devices without a depth sensor can be collected by devices that have one. Knowing the camera is no substitute: Depth Pro with the true focal length reaches 155.4 cm.
- 3) **It is not yet a sensor.** Through the same pipeline iPad LiDAR costs 3.0 cm, monocular depth with the correct scale per frame 5.3 cm, and a per-frame fit on 200 sparse metric points—what a phone’s tracker provides—6.0 cm. R3 (recall@10 cm 0.51) is still short of the “rough overview” tier (0.9), and further from furniture placement.
- 4) **The remaining error is a scale that swings per view, not shape:** one calibration per scene gives 22.9 ± 2.1 cm

in every room; the model’s per-frame scale varies severalfold within a scan, uncorrelated with time but correlated with what is in the frame. Fine-tuning removes the domain bias of that scale; the per-frame factor still needs metric data at run time.

- 5) **Secondary costs are small:** the pipeline itself loses 2.1 ± 0.5 cm at 4 cm voxels; DA-v2 Small costs 1.0 cm for a $6\times$ speed-up; ≥ 0.8 fps suffices for a complete room; LiDAR confidence weighting changes nothing.

The systems result: separating `DepthSource` and `ScaleAligner` into interfaces made all experiments pure YAML and made each fine-tuned checkpoint a new row in an existing table; the orchestrator was edited once in the whole project (two lines for confidence weights). The same code, behind a small upload/queue/view service and a loader for the app’s capture folder, is the product’s backend. Next steps: a controlled teacher comparison on identical frames, a larger validation set, fine-tuning data from more phone cameras, and combining the fine-tuned model with real ARKit/ARCore feature points to fix the per-frame scale it leaves.

REFERENCES

- [1] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2022.
- [2] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, “Depth anything: Unleashing the power of large-scale unlabeled data,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 10 371–10 381.
- [3] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth anything V2,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
- [4] S. F. Bhat, R. Birkel, D. Wofk, P. Wonka, and M. Müller, “ZoeDepth: Zero-shot transfer by combining relative and metric depth,” *arXiv preprint arXiv:2302.12288*, 2023.
- [5] W. Yin, C. Zhang, H. Wang, S. Niklaus, L. Mai, S. Chen, and C. Shen, “Metric3D: Towards zero-shot metric 3D prediction from a single image,” in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 9043–9053.
- [6] L. Piccinelli, Y.-H. Yang, C. Sakaridis, M. Segu, S. Li, L. Van Gool, and F. Yu, “UniDepth: Universal monocular metric depth estimation,” in *CVPR*, 2024.
- [7] A. Bochkovskii, A. Delaunoy, H. Germain, M. Santos, Y. Zhou, S. R. Richter, and V. Koltun, “Depth pro: Sharp monocular metric depth in less than a second,” in *ICLR*, 2025.
- [8] G. Baruch, Z. Chen, A. Dehghan, T. Dimry, Y. Feigin, P. Fu, T. Gebauer, B. Joffe, D. Kurz, A. Schwartz, and E. Shulman, “ARKitScenes: A diverse real-world dataset for 3D indoor scene understanding using mobile RGB-D data,” in *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [9] B. Curless and M. Levoy, “A volumetric method for building complex models from range images,” in *Proc. SIGGRAPH*, 1996, pp. 303–312.
- [10] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon, “Kinect-Fusion: Real-time dense surface mapping and tracking,” in *Proc. IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011, pp. 127–136.
- [11] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A modern library for 3D data processing,” *arXiv preprint arXiv:1801.09847*, 2018.
- [12] Z. Murez, T. van As, J. Bartolozzi, A. Sinha, V. Badrinarayanan, and A. Rabinovich, “Atlas: End-to-end 3D scene reconstruction from posed images,” in *Proc. European Conference on Computer Vision (ECCV)*, 2020, pp. 414–431.

- [13] J. Sun, Y. Xie, L. Chen, X. Zhou, and H. Bao, “NeuralRecon: Real-time coherent 3D reconstruction from monocular video,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 15 598–15 607.
- [14] M. Sayed, J. Gibson, J. Watson, V. Prisacariu, M. Firman, and C. Godard, “SimpleRecon: 3D reconstruction without 3D convolutions,” in *Proc. European Conference on Computer Vision (ECCV)*, 2022, pp. 1–19.
- [15] K. Tateno, F. Tombari, I. Laina, and N. Navab, “CNN-SLAM: Real-time dense monocular SLAM with learned depth prediction,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6243–6252.
- [16] Z. Yu, S. Peng, M. Niemeyer, T. Sattler, and A. Geiger, “MonoSDF: Exploring monocular geometric cues for neural implicit surface reconstruction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [17] Apple Inc., “RoomPlan – apple developer documentation,” <https://developer.apple.com/documentation/roomplan>, 2022, accessed 2026-09-17.

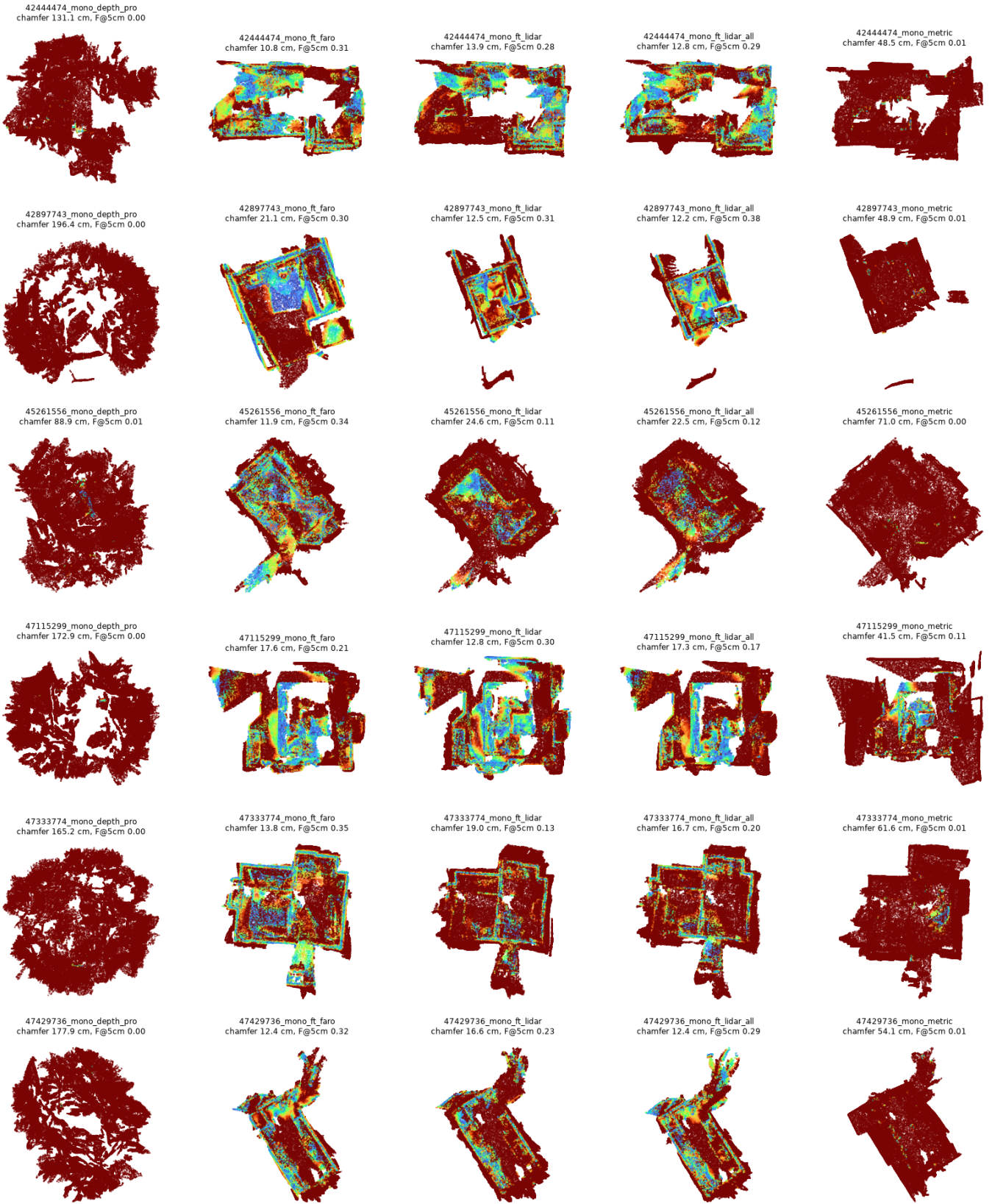


Fig. 3. Exp. 6 top-down error maps (rows = scenes, columns = Depth Pro, the three fine-tunes and the pretrained metric model, as labelled above each panel; colour = distance to the reference, 0–10 cm). The pretrained column is red almost everywhere—the room is built, at the wrong distance. After fine-tuning the walls and floor come back (blue/cyan) and the error concentrates at object edges and far walls; where a fine-tune is weaker (45261556 and 47333774 with the LiDAR teacher) it shows as a thick red rim around the room rather than a displaced room.

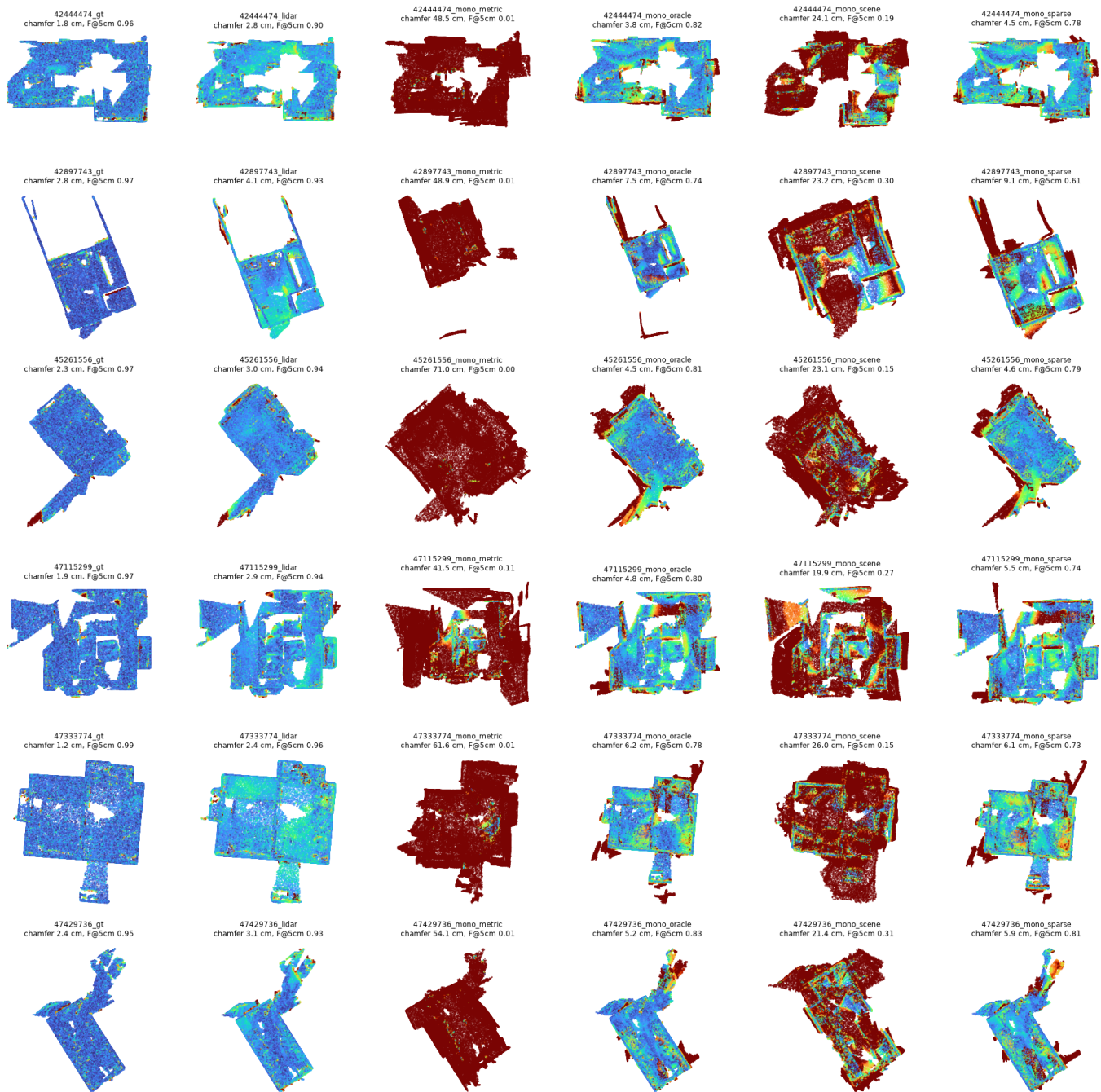


Fig. 4. Exp. 1 top-down error maps (rows = scenes, columns = runs, colour = distance to reference, 0–10 cm). The gt/lidar columns are almost entirely blue (<2 cm); oracle and sparse show the same yellow–red pattern at object edges and far walls; per_scene has a red blob in the middle of every room (phantom surfaces from wrongly scaled frames); metric is red almost everywhere.

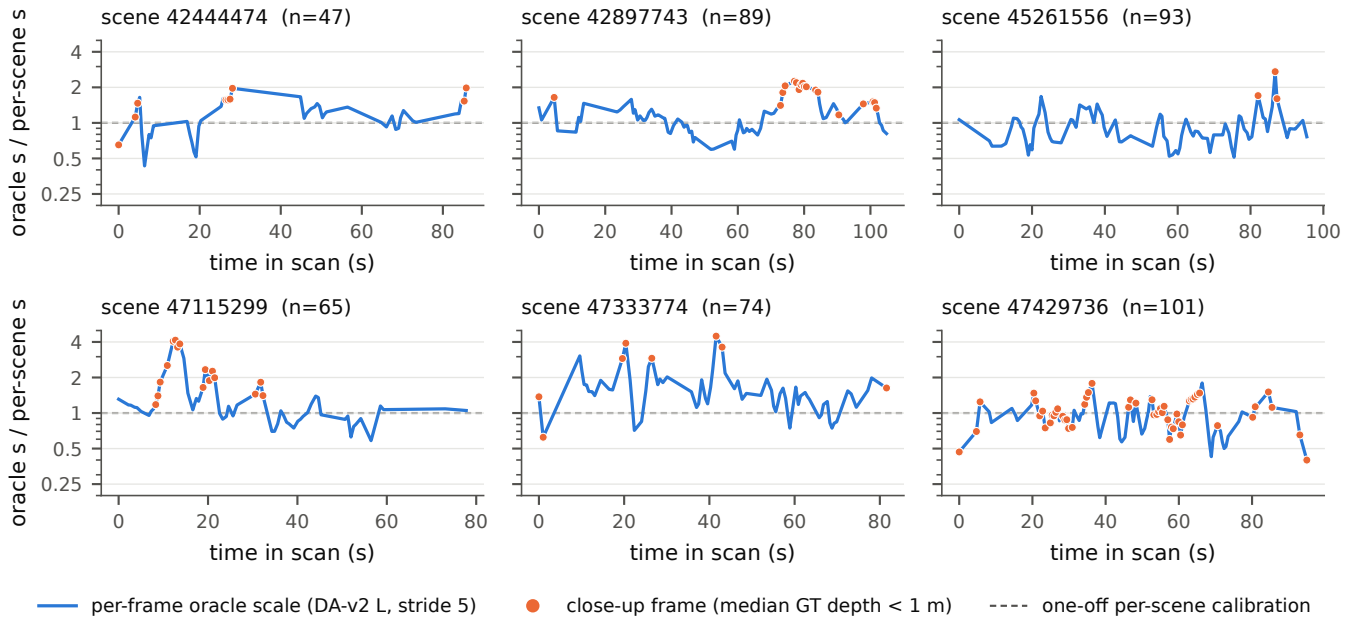


Fig. 5. Per-frame oracle scale divided by the per-scene scale, over time, for DA-v2 Large on all six scenes (log axis; 1.0 = the one-off calibration was right for that frame). Orange dots mark close-up frames (median GT depth < 1 m). The ratio swings by 2–4 $\times$ within every scan with no monotone trend.

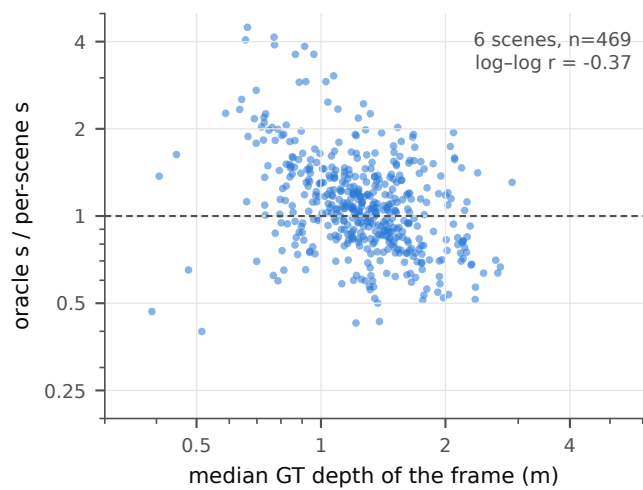


Fig. 6. The same ratio against the frame's median GT depth, all scenes pooled: the scale a frame needs correlates with how close the camera is to what it sees.